

DOCUMENTATION TECHNIQUE

VITE & GOURMAND

PROJET

Vite & Gourmand

TYPE DE PROJET

Application web de gestion de commandes pour une entreprise de traiteur

REALISE DANS LE CADRE

TP Développeur Web & Web Mobile

AUTEUR

BRUN Eline

DATE

22/07/2026

SOMMAIRE

1. Présentation du projet
2. Réflexion initiale & choix technologiques
3. Architecture de l'application
4. Gestion des données
5. Modèle conceptuel de données (MCD)
6. Diagrammes UML
 - Diagramme de cas d'utilisation
 - Diagramme de classes
 - Diagramme de séquence
7. Configuration de l'environnement de travail
8. Sécurité de l'application
9. Déploiement de l'application
10. Conclusion

1. PRESENTATION DU PROJET

Vite & Gourmand est une application web développée pour une entreprise de traiteur située à Bordeaux.

L'objectif du projet est de proposer une plateforme permettant aux clients de consulter les différents menus proposés & de réaliser des prises de commandes en lignes.

L'application permet également au personnel interne de gérer l'activité quotidienne grâce à des espaces dédiés aux employés et aux administrateurs.

Les objectifs du projet :

- Proposer une interface accessible aux clients souhaitant commander des prestations culinaires ;
- Automatiser la gestion des commandes ;
- Centraliser les informations relatives aux menus, utilisateurs & stocks ;
- Faciliter le suivi administratif grâce à un espace dédié ;
- Exploiter des données statistiques afin d'aider à la prise de décision.

Fonctionnalités principales :

Espace client – l'utilisateur peut :

- Créer un compte ;
- Se connecter à son espace personnel ;
- Consulter les menus disponibles ;
- Filtrer les menus selon différents critères
- Ajouter des menus à son panier
- Passer une commande
- Bénéficier d'un calcul automatique de frais de livraison selon l'adresse renseignée
- Suivre l'état d'avancement d'une commande
- Publier un avis
- Consulter son historique de commande
- Consulter le récapitulatif des commandes passées

- Modifier ses informations personnelles ainsi que son mot de passe

Espace employé - l'employé peut :

- Consulter les commandes
- Filtrer les commandes
- Modifier l'état des commandes
- Gérer les menus
- Créer un nouveau menu
- Gérer les stocks
- Gérer les avis clients
- Modifier son mot de passe

Espace administrateur – l'administrateur peut :

- Consulter les statistiques commerciales sous forme de cartes (CA & nombre de commandes)
- Consulter les statistiques commerciales sous forme de graphique (nombre de commandes par menu)
- Consulter les commandes
- Filtrer les commandes
- Modifier l'état des commandes
- Gérer les employés
- Créer un employé
- Gérer les menus
- Créer un nouveau menu
- Gérer les stocks
- Gérer les avis clients
- Consulter les messages reçus
- Modifier son mot de passe

Architecture générale :

L'application repose sur une architecture trois tiers :

Utilisateur

↓

Interface Web

(HTML / CSS / JavaScript / Bootstrap)

↓
Serveur applicatif
(PHP)
↓
Bases de données
(MariaDB + MongoDB)

Base NoSQL MongoDB

Utilisée pour les données statistiques.

Elle permet notamment de stocker les informations nécessaires à l'affichage des graphiques :

- Nombre de commandes par menu ;
- Chiffre d'affaires ;
- Evolution des ventes.

Ces données sont ensuite exploitées avec Chart.js.

2. REFLEXION INITIALE & CHOIX TECHNOLOGIQUES

Lors de la conception de l'application Vite & Gourmand, l'objectif principal était de développer une plateforme web complète permettant de répondre aux besoins de différents types d'utilisateurs :

- Les clients souhaitant commander des prestations traiteur ;
- Les employés chargés du suivi opérationnel des commandes ;
- Les administrateurs responsables de la gestion globale de l'activité.

Le choix de l'architecture & des technologies a été réalisé afin de répondre à plusieurs contraintes :

- Proposer une application responsive accessible depuis différents supports ;
- Garantir la sécurité des données utilisateurs ;
- Permettre une gestion fiable des informations métier ;
- Faciliter la maintenance & l'évolution future de l'application ;
- Respecter les bonnes pratiques de développement web.

L'application repose donc sur une architecture trois tiers, séparant :

- La présentation (interface utilisateur) ;
- La logique applicative côté serveur ;
- La gestion des données.

Cette séparation permet une meilleure organisation du projet & facilite les évolutions futures.

3. ARCHITECTURE DE L'APPLICATION

Architecture trois tiers

L'application est organisée selon les trois couches suivantes :

- A. Couche de présentation - correspond à l'interface visible par les utilisateurs

Elle regroupe :

- Les pages HTML ;
- Les feuilles de style CSS ;
- Le framework Bootstrap ;
- Les scripts JavaScript.

Son rôle est de :

- Afficher les informations ;
- Permettre les interactions utilisateurs ;
- Transmettre les actions réalisées vers le serveur.

Exemple :

- Affichage des menus ;
- Gestion dynamique des filtres ;
- Calcul du panier ;
- Calcul de la remise ;
- Calcul de la livraison.

- B. Couche logique métier

Cette couche correspond au traitement effectué côté serveur. Elle est développée en PHP & assure notamment :

- L'authentification des utilisateurs ;
- La gestion des sessions ;
- La validation des formulaires ;
- Le traitement des commandes ;
- Le calcul des frais de livraison ;
- La gestion des droits utilisateurs ;
- Les échanges avec les bases de données.

Cette couche contient la logique métier de l'application.

Exemple :

Lorsqu'un client valide une commande :

1. Le serveur vérifie que l'utilisateur est connecté ;
2. Les informations du panier sont contrôlées ;
3. Les règles métier sont appliquées :
 - Disponibilité des menus ;
 - Nombre minimum de personnes ;
 - Remise éventuelle
 - Frais de livraison.
4. La commande est enregistrée en base de données ;
5. Une confirmation par mail est envoyée au client.

C. Couche données

Cette couche correspond au stockage & à la gestion des informations.

Deux systèmes de stockages sont utilisés :

- MariaDB pour les données métier ;
- MongoDB pour les statistiques.

Cette séparation permet d'utiliser une technologie adaptée à chaque besoin.

Choix des technologies :

FRONT-END

HTML5 a été choisi pour structurer les différentes pages de l'application.

Il permet notamment :

- Une organisation claire du contenu ;
- Une meilleure accessibilité ;
- Une compatibilité avec les navigateurs modernes.

CSS & Bootstrap

Le CSS permet de personnaliser l'apparence graphique de l'application.

Le framework Bootstrap a été utilisé afin de :

- Accélérer le développement de l'interface ;
- Garantir une compatibilité responsive ;
- Adapter l'affiche aux différents formats d'écran :
- Ordinateur ;
- Tablette ;
- Mobile.

Javascript

Javascript est utilisé pour rendre l'application interactive.

Il permet notamment :

- a. La mise à jour dynamique des filtres menus ;
- b. Les calculs du panier ;
- c. La récupération automatique des frais de livraison ;
- d. Les échanges asynchrones avec le serveur ;
- e. Les horaires d'ouverture et de fermeture.

Exemple :

Le calcul des frais de livraison utilise l'API Google Maps afin d'obtenir la distance entre l'adresse du restaurant & celle du client.

BACK-END

PHP a été choisi pour développer la partie serveur de l'application.

- Ce langage permet de gérer :
- La communication avec les bases de données ;
- L'authentification ;

- Les traitements métier ;
- La génération dynamique des pages.

PHP a également été retenu pour sa compatibilité avec les solutions d'hébergement utilisées.

4.GESTION DES DONNEES

Base de données relationnelle MariaDB

MariaDB est utilisée pour stocker les données principales de l'application.

Le choix d'une base relationnelle est adapté aux données métier nécessitant des relations entre plusieurs entités.

Elle permet de gérer :

- Les utilisateurs (users) ;
- Les rôles (employees) ;
- Les menus (menus) ;
- Les plats (menu_items) ;
- Les commandes (orders) ;
- Les détails de commandes (orders_items) ;
- Les stocks (stocks) ;
- Les allergènes (menus_allergens) ;
- Les régimes (diets) ;
- Les thèmes (themes) ;
- Le formulaire de contact (contact_messages) ;
- L'oubli de mot de passe (password_resets) ;
- Les avis clients (reviews).

Les relations entre les tables permettent de garantir la cohérence des données grâce :

- Aux clés primaires ;
- Aux clés étrangères ;
- Aux contraintes d'intégrité.

L'accès à la base est réalisé avec PHP PDO afin d'utiliser des requêtes préparées & limiter les risques d'injection SQL.

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ allergens	★ Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ contact_messages	★ Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_general_ci	16,0 kio	-
☐ diets	★ Parcourir Structure Rechercher Insérer Vider Supprimer	3	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ employees	★ Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ menus	★ Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ menus_allergens	★ Parcourir Structure Rechercher Insérer Vider Supprimer	35	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ menu_items	★ Parcourir Structure Rechercher Insérer Vider Supprimer	33	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ orders_items	★ Parcourir Structure Rechercher Insérer Vider Supprimer	9	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ orders	★ Parcourir Structure Rechercher Insérer Vider Supprimer	6	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ password_resets	★ Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ reviews	★ Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ stocks	★ Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ themes	★ Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ users	★ Parcourir Structure Rechercher Insérer Vider Supprimer	7	InnoDB	utf8mb4_general_ci	32,0 kio	-
14 tables	Somme	128	InnoDB	latin1_swedish_ci	512,0 kio	0 o

Base de données NoSQL MongoDB

MongoDB est utilisée en complément de MariaDB pour la gestion des données statistiques.

Contrairement aux données métier, les statistiques évoluent régulièrement & nécessitent une structure plus flexible.

MongoDB permet de stocker des documents contenant notamment :

- Les menus concernés ;
- Le nombre de commandes ;
- Le chiffre d'affaires généré ;
- Les données nécessaires aux graphiques.

Ces informations sont ensuite exploitées par Chart.js afin de générer des représentations graphiques dans l'espace administrateur.

Le choix d'une base NoSQL permet donc :

- Une plus grande flexibilité dans la structure des données ;
- Une récupération rapide des statistiques ;
- Une séparation claire entre données métier et données analytiques.

menu_statistics vite_gourmand +

localhost:27017 > vite_gourmand > menu_statistics [Open MongoDB shell](#)

Documents 11 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) [Explain](#) [Reset](#) [Find](#) [Options](#)

[ADD DATA](#) [BULK](#) [EXPORT DATA](#) [EXPORT CODE](#) 25 1 - 11 of 11

```
{
  "_id": ObjectId('6a59750b45b502787b09ba36'),
  "menu_id": 1,
  "menu_name": "MENU DU MOIS",
  "total_orders": 1,
  "total_revenue": 25
}
```

```
{
  "_id": ObjectId('6a59750b45b502787b09ba37'),
  "menu_id": 2,
  "menu_name": "MENU DE LA MAMA",
  "total_orders": 2,
  "total_revenue": 150
}
```

```
{
  "_id": ObjectId('6a59750b45b502787b09ba38'),
  "menu_id": 3,
  "menu_name": "MENU DE LA MER",
  "total_orders": 0,
  "total_revenue": 0
}
```

```
{
  "_id": ObjectId('6a59750b45b502787b09ba39'),
  "menu_id": 4,
  "menu_name": "MENU SIGNATURE",
  "total_orders": 0,
  "total_revenue": 0
}
```

```
{
  "_id": ObjectId('6a59750b45b502787b09ba3a'),
  "menu_id": 5,
  "menu_name": "MENU VÉGÉTARIEN",
  "total_orders": 1,
  "total_revenue": 60
}
```

Services externes utilisés API Google Maps

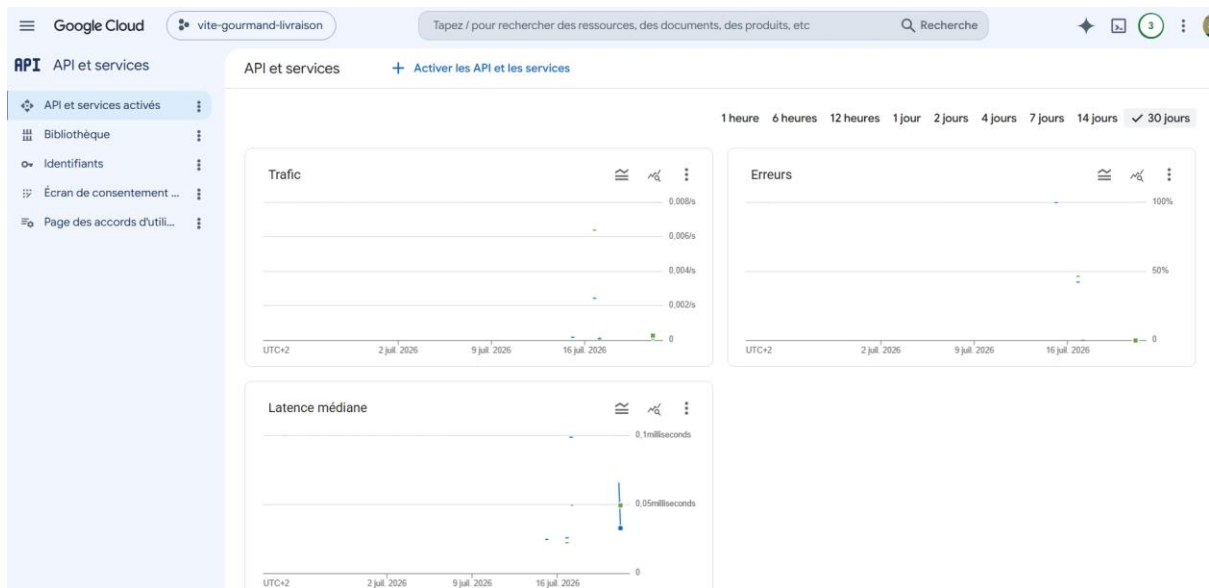
L'application utilise l'API Google Maps pour calculer automatiquement les frais de livraison.

Elle permet :

- La saisie assistée d'une adresse ;
- Le calcul de distance entre le restaurant et le client ;
- L'application des règles tarifaires définies par l'entreprise.

La logique métier appliquée est :

- Livraison gratuite pour Bordeaux ;
- Forfait de base de 5 € ;
- Supplément de 0,59 €/km au-delà de la zone définie.



Outils de développement

Les outils utilisés pour réaliser le projet sont :

VISUAL STUDIO CODE	DEVELOPPEMENT
GITHUB	GESTION CODE SOURCE
GIT	GESTION VERSIONS
FIGMA	MAQUETTES
TRELLO	GESTION TACHES
XAMPP	ENVIRONNEMENT LOCAL
PHPMYADMIN	GESTION MARIADB
MONGODB COMPASS	GESTION NOSQL
COMPOSER	GESTION DES DEPENDANCES

5.MODELE CONCEPTUEL DE DONNEES MCD

Présentation

Le modèle conceptuel de données (MCD) représente les principales entités manipulées par l'application ainsi que les relations existantes entre elles.

Il a été conçu afin de garantir une organisation cohérente des données et de limiter les redondances grâce à la normalisation de la base relationnelle.

La base de données repose principalement sur les entités suivantes :

- Les utilisateurs (users) ;
- Les rôles (employees) ;
- Les menus (menus) ;
- Les plats (menu_items) ;

- Les commandes (orders) ;
- Les détails de commandes (orders_items) ;
- Les stocks (stocks) ;
- Les allergènes (menus_allergens) ;
- Les régimes (diets) ;
- Les thèmes (themes) ;
- Le formulaire de contact (contact_messages) ;
- L'oubli de mot de passe (password_resets) ;
- Les avis clients (reviews).

Les relations entre ces entités permettent de représenter l'ensemble du fonctionnement de l'application.

Description des principales entités

UTILISATEUR

Cette entité regroupe les informations relatives aux clients de l'application.

Elle contient notamment :

- Nom ;
- Prénom ;
- Coordonnées ;
- Email ;
- Téléphone ;
- Adresse ;
- Mot de passe ;
- Rôle associé.

Un utilisateur peut effectuer plusieurs commandes.

COMMANDE

La commande représente une réservation réalisée par un client.

Elle contient notamment :

- La date de commande & celle de livraison ;
- Le statut ;
- Le montant ;
- Les frais de livraison ;
- Les remises appliquées.

Une commande appartient à un seul utilisateur mais peut contenir plusieurs menus.

MENU

Cette entité représente les différentes prestations proposées par le traiteur.

Chaque menu possède notamment :

- Un nom ;
- Une description ;
- Un prix ;
- Un nombre minimum de personnes ;
- Un thème ;
- Un régime alimentaire ;
- Un délai de commande ;
- Une image principale.

Un menu est composé de plusieurs plats.

PLAT

Les plats constituent les éléments composant un menu.

Ils peuvent être associés à plusieurs menus.

Cette relation est gérée grâce à une table d'association.

Chaque plat possède notamment :

- Un nom ;
- Une description ;
- Une image.

STOCK

Le stock permet de suivre la disponibilité des menus.

Il est mis à jour lors des commandes.

AVIS

Les utilisateurs peuvent laisser un avis après la réalisation d'une commande.

Chaque avis est associé à une commande afin de garantir que seuls les clients ayant effectivement commandé puissent déposer une évaluation.

RÔLE

Les rôles permettent de gérer les droits d'accès de l'application.

Trois profils sont définis :

- Administrateur
- Employé
- Client

Relations principales

Les principales relations du modèle sont les suivantes :

- Un rôle est associé à plusieurs utilisateurs ;
- Un utilisateur peut effectuer plusieurs commandes ;
- Une commande contient plusieurs menus ;
- Un menu peut être présent dans plusieurs commandes ;
- Un menu est composé de plusieurs plats ;
- Un plat peut appartenir à plusieurs menus ;
- Un menu peut être associé à plusieurs allergènes ;
- Un allergène peut concerner plusieurs menus ;
- Un régime alimentaire peut être associé à plusieurs menus ;
- Un thème peut être associé à plusieurs menus ;
- Une commande peut recevoir un seul avis.

CARDINALITES

ENTITE A	CARDINALITE	ENTITE B
themes	1->N	menus
diets	1->N	menus
menu	1->N	menu_items
menu	1->1	stocks
users	1->N	orders
menu	1->N	oders_items
orders	1->N	oders_items
users	1->N	reviews
orders	1->0..1	reviews
users	1->0..1	employee
users	1->N	password_resets
menu	N->N	allergens via menus_allergens

6.DIAGRAMMES UML

Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les différentes interactions entre les acteurs de l'application et les fonctionnalités qui leur sont accessibles.

Trois acteurs principaux ont été identifiés :

- Client
- Employé

→ Administrateur

Chaque acteur dispose de droits spécifiques correspondant à son rôle au sein de l'application.

Cas d'utilisation du client

Le client peut :

- Créer un compte
- Se connecter
- Réinitialiser son mot de passe
- Consulter les menus
- Rechercher et filtrer les menus
- Consulter le détail d'un menu
- Ajouter un menu au panier
- Modifier son panier
- Calculer les frais de livraison
- Passer une commande
- Consulter son historique de commandes
- Suivre l'état d'une commande
- Modifier son profil
- Déposer un avis
- Contacter le traiteur via le formulaire de contact

Cas d'utilisation de l'Employé

L'employé peut :

- Se connecter
- Consulter son planning
- Consulter les commandes
- Modifier le statut d'une commande
- Consulter les informations clients nécessaires à la livraison
- Consulter les stocks

Cas d'utilisation de l'Administrateur

L'administrateur dispose de l'ensemble des fonctionnalités de gestion.

Il peut notamment :

- Se connecter
- Gérer les menus (création, modification, suppression)
- Gérer les stocks
- Gérer les employés
- Consulter les messages de contact
- Consulter les commandes
- Modifier le statut des commandes
- Valider ou refuser les avis
- Consulter les statistiques issues de MongoDB
- Gérer les comptes utilisateurs

Diagramme UML

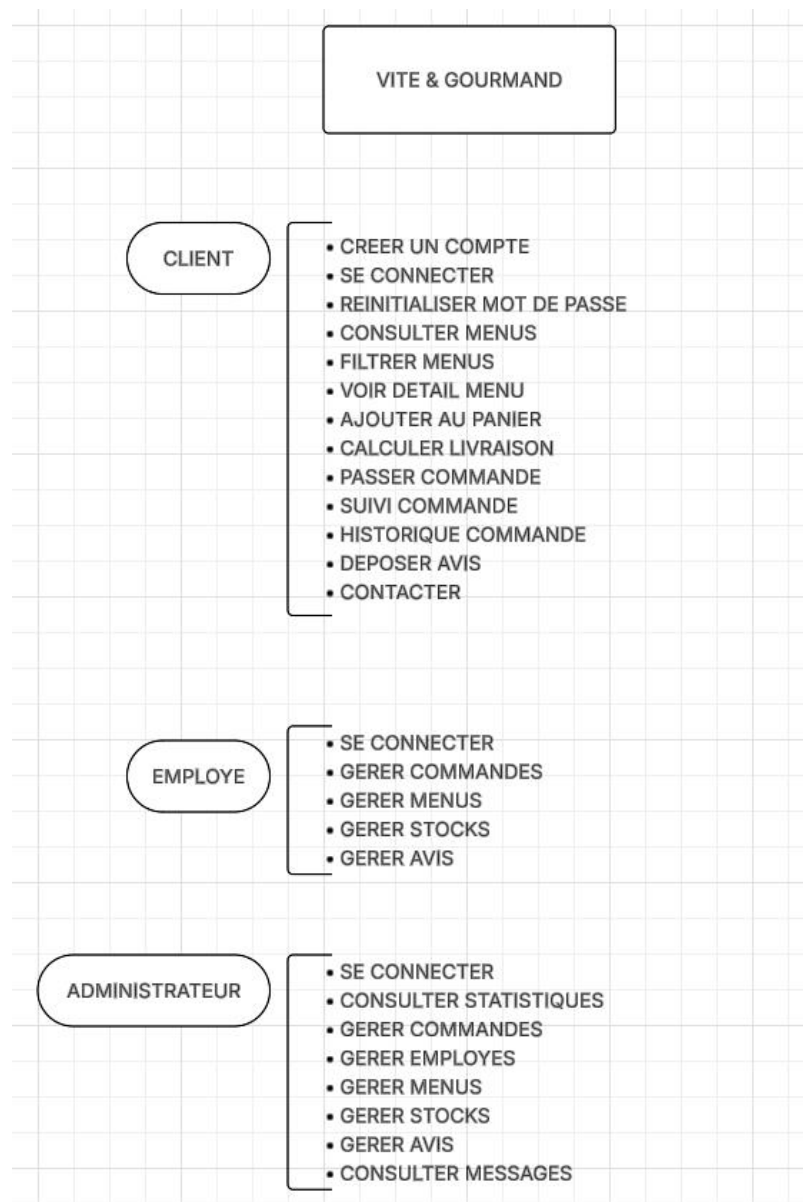


Diagramme de classes

Le diagramme de classes représente les principales classes métier de l'application ainsi que les relations existantes entre elles.

Contrairement au MCD, qui décrit les données de manière conceptuelle, le diagramme de classes met davantage l'accent sur les objets manipulés par l'application et leurs associations.

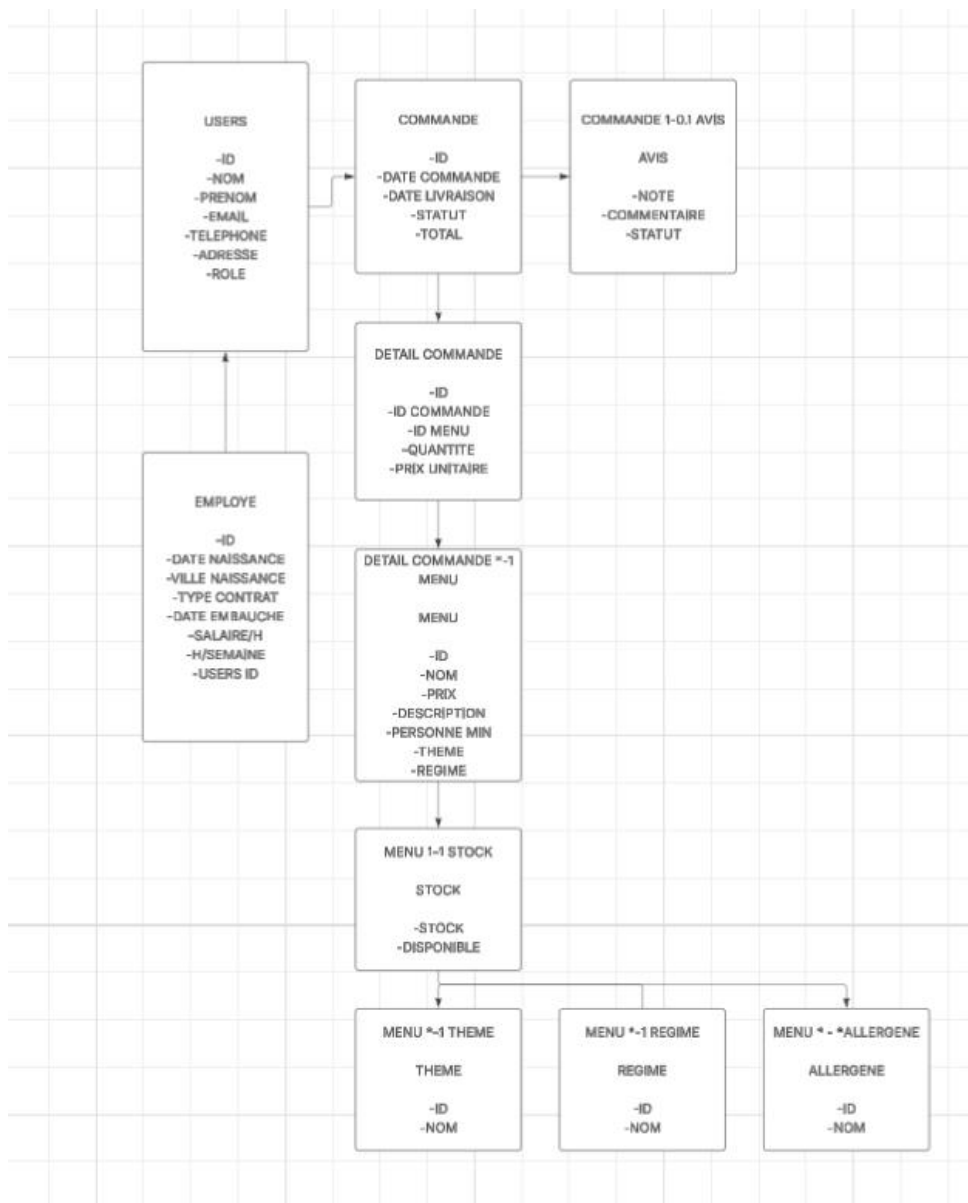
Les classes représentées correspondent directement aux principales entités utilisées par le système de gestion de Vite & Gourmand.

Les principales classes

Les classes principales sont :

- Les utilisateurs (users) ;
- Les rôles (employees) ;
- Les menus (menus) ;
- Les plats (menu_items) ;
- Les commandes (orders) ;
- Les détails de commandes (orders_items) ;
- Les stocks (stocks) ;
- Les allergènes (menus_allergens) ;
- Les régimes (diets) ;
- Les thèmes (themes) ;
- Les avis clients (reviews).

Diagramme UML



Le diagramme de classes présente les principales entités métier de l'application ainsi que leurs relations. Cette représentation facilite la compréhension de l'architecture fonctionnelle de l'application et constitue une base pour les évolutions futures.

Diagramme de séquence

Le diagramme de séquence permet de représenter chronologiquement les échanges entre les différents composants de l'application lors de l'exécution d'un scénario.

Il met en évidence les interactions entre :

- L'utilisateur ;

- L'interface web ;
- Le serveur PHP ;
- La base de données MariaDB ;
- La base MongoDB lorsque cela est nécessaire.

Deux scénarios représentatifs sont présentés :

- Authentification d'un utilisateur ;
- Validation d'une commande.

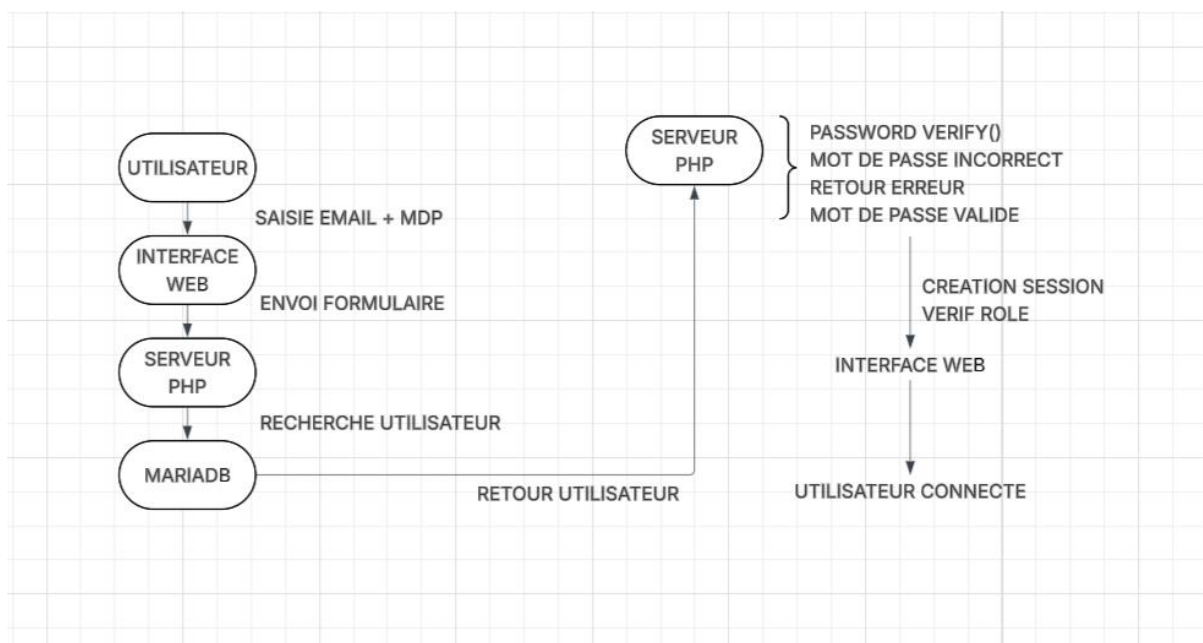
Diagramme n°1 — Authentification

Description

Ce diagramme illustre le processus d'authentification d'un utilisateur.

Lorsqu'un utilisateur renseigne son adresse électronique et son mot de passe, le serveur vérifie les informations enregistrées dans la base de données avant d'autoriser l'accès à l'application.

Le mot de passe n'est jamais comparé en clair. La vérification est réalisée à l'aide de la fonction `password_verify()`.



Ce qu'il montre

Il met en évidence :

- La recherche du compte utilisateur ;
- La vérification du mot de passe haché ;
- La création de la session ;
- La redirection vers l'espace correspondant au rôle de l'utilisateur.

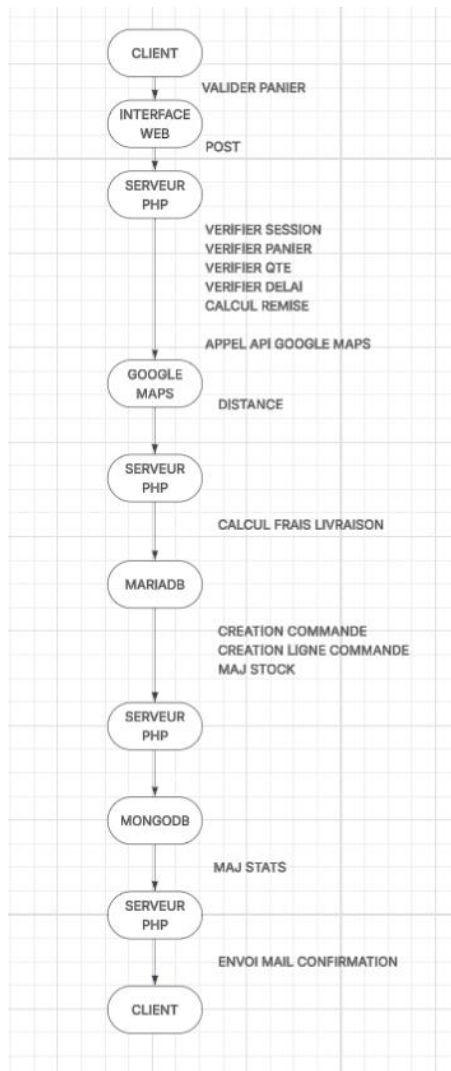
Diagramme n°2 — Passage d'une commande

Description

Ce scénario représente le traitement complet d'une commande depuis sa validation par le client jusqu'à son enregistrement en base de données.

Plusieurs contrôles sont réalisés avant la création définitive de la commande :

- Utilisateur connecté ;
- Disponibilité des menus ;
- Calcul des remises ;
- Calcul des frais de livraison via l'API Google Maps ;
- Enregistrement des données ;
- Mise à jour des statistiques MongoDB ;
- Envoi du courrier électronique de confirmation.



Règles métier

Les principales règles métier implémentées dans l'application sont les suivantes :

- Authentification obligatoire avant toute commande ;
- Respect du nombre minimum de personnes pour chaque menu ;
- Application automatique d'une remise de 10 % lorsque le nombre de personnes dépasse de cinq le minimum requis ;
- Calcul automatique des frais de livraison à partir de la distance entre le restaurant et l'adresse du client ;
- Livraison gratuite pour les commandes situées à Bordeaux ;
- Contrôle du délai minimal de préparation propre à chaque menu ;
- Interdiction de sélectionner une date de livraison incompatible avec les délais de préparation ou les jours de fermeture ;
- Un avis est associé à une commande afin de garantir que seuls les clients ayant effectivement commandé puissent déposer une évaluation.

7. CONFIGURATION DE L'ENVIRONNEMENT DE TRAVAIL

Présentation

Le développement de l'application Vite & Gourmand a été réalisé dans un environnement local avant son déploiement sur un hébergement web.

L'ensemble des outils utilisés a été sélectionné afin de faciliter le développement, les tests, la gestion des bases de données et le suivi du projet.

Environnement matériel

Le projet a été développé sur un poste de travail équipé du système d'exploitation Windows.

Les tests ont été réalisés en local à l'aide d'un serveur Apache fourni par XAMPP.

Logiciels utilisés

VISUAL STUDIO CODE	DEVELOPPEMENT
GITHUB	GESTION CODE SOURCE
GIT	GESTION VERSIONS
FIGMA	MAQUETTES
TRELLO	GESTION TACHES
XAMPP	ENVIRONNEMENT LOCAL
PHPMYADMIN	GESTION MARIADB
MONGODB COMPASS	GESTION NOSQL
COMPOSER	GESTION DES DEPENDANCES

Langages et technologies

Les technologies utilisées sont les suivantes :

Front-end

- HTML5
- CSS3

- Bootstrap
- JavaScript

Back-end

- PHP 8

Bases de données

- MariaDB
- MongoDB

API externe

- Google Maps (calcul des distances et des frais de livraison)

Installation locale du projet

L'installation de l'application nécessite les étapes suivantes :

1. Installer XAMPP et démarrer les services Apache et MariaDB.
2. Cloner le dépôt GitHub.
3. Installer les dépendances PHP à l'aide de Composer.
4. Importer la base de données MariaDB.
5. Configurer le fichier config.php avec les paramètres de connexion et la clé API Google Maps.
6. Vérifier la connexion à MongoDB.
7. Accéder à l'application via le serveur local.

8. SECURITE DE L'APPLICATION

Présentation

La sécurité constitue un élément essentiel de l'application afin de garantir la confidentialité des données personnelles et la protection des fonctionnalités sensibles.

Plusieurs mécanismes ont été mis en œuvre pour limiter les risques liés aux attaques les plus courantes.

Authentification

L'accès aux espaces privés nécessite une authentification.

Chaque utilisateur possède un compte personnel comprenant :

- Une adresse électronique unique ;
- Un mot de passe sécurisé ;
- Un rôle (client, employé ou administrateur).

Les droits d'accès sont adaptés en fonction du rôle de l'utilisateur connecté.

Protection des mots de passe

Les mots de passe ne sont jamais stockés en clair dans la base de données.

Lors de la création d'un compte ou d'une modification de mot de passe, ceux-ci sont chiffrés à l'aide de la fonction `password_hash()` de PHP.

Lors de la connexion, la vérification est réalisée grâce à la fonction `password_verify()`.

Cette méthode garantit qu'aucun mot de passe lisible n'est conservé dans la base de données.

Protection contre les injections SQL

Toutes les interactions avec MariaDB sont réalisées à l'aide de PDO et de requêtes préparées.

Cette approche permet de séparer les données fournies par l'utilisateur des instructions SQL, réduisant ainsi le risque d'injection SQL.

Gestion des sessions

Après authentification, une session PHP est créée.

Les informations de session permettent notamment de conserver :

- L'identité de l'utilisateur ;

- Son rôle ;
- Son état de connexion.

Les pages protégées vérifient systématiquement la présence d'une session valide avant d'autoriser l'accès.

Gestion des autorisations

L'application distingue trois profils :

- Client
- Employé
- Administrateur

Chaque espace est protégé afin que seules les fonctionnalités autorisées soient accessibles.

Par exemple :

- Un client ne peut pas accéder au tableau de bord d'administration ;
- Un employé ne peut pas modifier les comptes administrateurs ;
- Un administrateur dispose de l'ensemble des droits de gestion.

Protection des fichiers sensibles

Les informations sensibles, telles que les paramètres de connexion aux bases de données ou la clé API Google Maps, sont regroupées dans des fichiers de configuration dédiés.

Ces fichiers ne sont pas destinés à être publiés dans un dépôt GitHub public et sont exclus du versionnement afin de préserver la confidentialité des identifiants.

Validation des données

Les données saisies par les utilisateurs sont contrôlées avant tout traitement.

Les principales validations portent notamment sur :

- Les champs obligatoires ;

- Le format des adresses électroniques ;
- Les dates de livraison ;
- Les quantités commandées ;
- Le respect des règles métier (nombre minimum de personnes, délais de préparation, etc.).

9. DEPLOIEMENT DE L'APPLICATION

Présentation

L'application est destinée à être déployée sur un hébergement web mutualisé.

Le déploiement comprend le transfert des fichiers de l'application ainsi que la configuration des différentes bases de données et services nécessaires à son fonctionnement.

Étapes du déploiement

1. Création de l'hébergement.
2. Configuration du nom de domaine.
3. Dépôt des fichiers de l'application.
4. Importation de la base MariaDB.
5. Configuration des paramètres de connexion.
6. Tests fonctionnels de l'application.
7. Validation de la mise en production.

10. CONCLUSION

L'application Vite & Gourmand répond aux principaux besoins d'un service traiteur en proposant une solution complète de gestion des commandes, des utilisateurs et des menus.

Le projet combine une base de données relationnelle (MariaDB) pour les données métier et une base NoSQL (MongoDB) pour les statistiques, tout en intégrant des services externes tels que Google Maps pour le calcul automatisé des frais de livraison.

Le développement de cette application a permis de mettre en œuvre les compétences acquises au cours de la formation, notamment en matière de conception de bases de

données, de développement web, de sécurisation des données, de gestion de projet et de déploiement d'une application.

Des évolutions pourraient être envisagées à l'avenir, telles que l'ajout d'un système de paiement en ligne, de notifications en temps réel ou d'outils d'analyse statistique plus avancés.